AXI2APB Bridge: UVM Verification Environment Guide

Code walkthrough and design logic. Prepared October 19, 2026. UVM 1.2 on Synopsys VCS.

1. Purpose and scope

This guide explains the UVM environment built to verify the `axi2apb_wrap` bridge: what each file does, the logic inside it, and why it was written that way. It is meant for someone who has to read, run, extend or defend the testbench.

The environment contains:

- An active **AXI master agent** that drives write and read requests into the bridge.
- A reactive **APB slave agent** that answers the transfers the bridge starts, with controllable wait states, errors and read data, plus a passive monitor.
- A **virtual sequencer** that coordinates both agents from one sequence.
- **13 AXI sequences** written with `uvm_do_with`, one per area of the verification plan, and 3 APB response sequences.
- A **functional coverage** class fed by both agents.
- A set of tests that run the sequences.

Not built yet: a scoreboard, a true AXI monitor, and a reset-in-the-middle-of-traffic test. Section 15 lists these.

2. The DUT behaviour that shaped the design

The bridge is an AXI **slave** and an APB **master**. The testbench therefore plays the opposite roles: AXI master and APB slave. The facts below come from the verification plan, the bridge RTL and the waveforms.

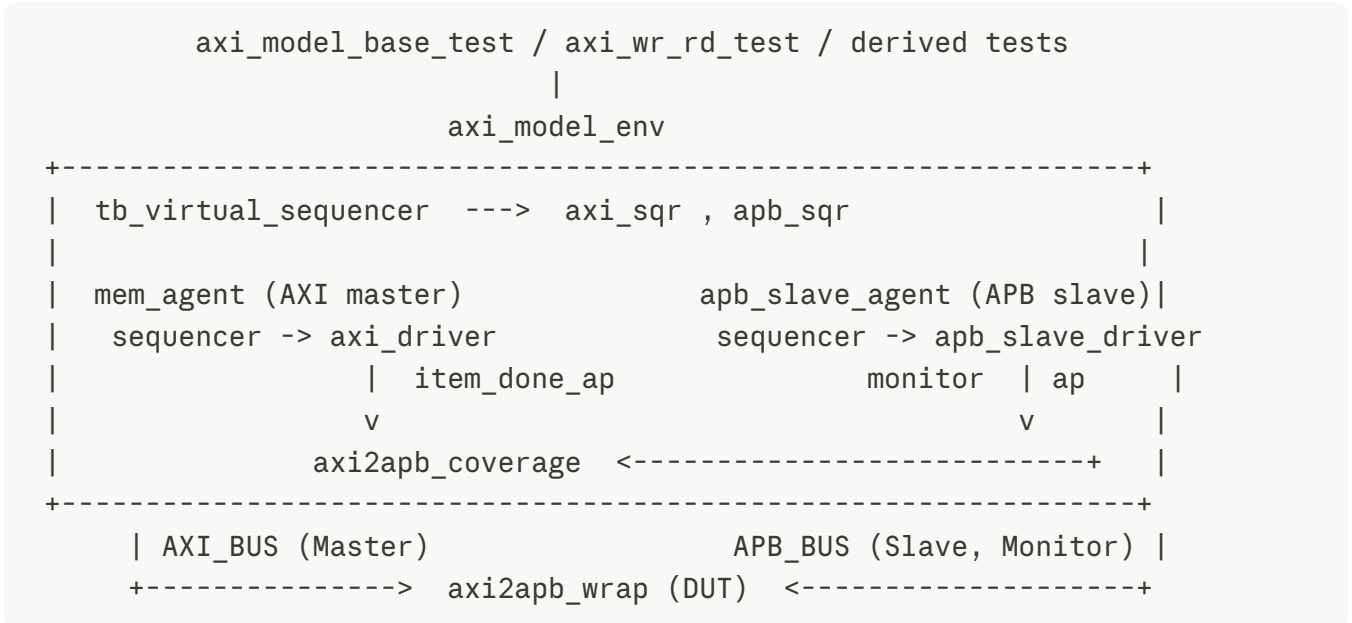
- **FSM states:** IDLE, WAIT_W_PREADY, SEND_B_RESP, WAIT_R_PREADY, DONE_SINGLE_RD.
- **Priority:** in IDLE, ARVALID wins over AWVALID.
- **Write acceptance:** a write starts only when a write address and write data are both valid together.

- **Waiting:** a write waits in WAIT_W_PREADY until PREADY is high, then goes to SEND_B_RESP and asserts BVALID. A read waits in WAIT_R_PREADY, samples PRDATA into RDATA_Q when PREADY is high, then goes to DONE_SINGLE_RD, which asserts RVALID and RLAST.
- **Responses:** BRESP and RRESP are always OKAY and PSLVERR is not used. BID/BUSER come from the captured AWID/AWUSER, and RID/RUSER from ARID/ARUSER.
- **Unused inputs:** WSTRB and WLAST are not used by the FSM. Only a single beat is converted. Burst fields exist, but no address progression is implemented.
- **Reset:** active-low and asynchronous. It forces the FSM to IDLE and clears RDATA_Q.
- **APB signalling (important):** the RTL assigns `PSEL = 1'b1` permanently and `PENABLE = write_req | read_req`. PENABLE therefore rises in the same cycle as PADDR, PWRITE and PWDATA. There is no APB setup phase (PSEL=1, PENABLE=0) as AMBA APB requires. PREADY is sampled in the first cycle, so a zero-wait-state transfer is possible.

Two consequences drove most of the design:

1. If nothing answers on the APB side, PREADY never rises, the FSM never reaches SEND_B_RESP, BVALID never comes, the AXI driver waits forever and the run ends with a UVM `PH_TIMEOUT`. The APB slave is therefore mandatory.
2. A conventional APB agent that waits for a setup phase would misread this bridge. The APB driver and monitor trigger on PENABLE instead.

3. Architecture



Data flow:

- A test starts a virtual sequence on `vsqr`. It starts sequences on the AXI sequencer and, when needed, an APB response sequence on the APB sequencer.
- The AXI driver drives `AXI_BUS` into the DUT. The DUT drives `APB_BUS`. The APB slave driver answers, and the APB monitor watches.
- When an AXI transaction completes, the driver broadcasts a copy on `item_done_ap`. When an APB transfer completes, the monitor broadcasts it on its analysis port. The coverage class subscribes to both.

4. Files, package and compile order

Files by area:

- **Interfaces:** `axi_bus.sv` (yours) and `apb_if.sv`.
- **AXI agent:** `axi_seq_item.sv`, `axi_sequencer.sv`, `driver.sv`, `axi_agent.sv` (class `mem_agent`), `axi_sequence.sv`.
- **APB agent:** `apb_seq_item.sv`, `apb_slave_cfg.sv`, `apb_sequencer.sv`, `apb_slave_driver.sv`, `apb_monitor.sv`, `apb_slave_agent.sv`, `apb_slave_sequences.sv`.
- **Environment:** `tb_virtual_sequencer.sv`, `tb_virtual_sequences.sv`, `axi2apb_coverage.sv`, `axi_env.sv`.
- **Tests:** `axi_base_test.sv`, `axi_wr_rd_test.sv`, `axi_apb_tests.sv`.
- **Top:** `mem_tb_uvm_pkg.sv` and `testbench.sv`.

The package `mem_tb_uvm_pkg` includes every class file in dependency order: AXI agent, APB agent, sequences, virtual sequencer and sequences, coverage, env, tests. A class must come after any class it uses.

Compile order in `build.f`: interfaces, DUT files, `mem_tb_uvm_pkg.sv`, then `testbench.sv`. Interfaces come before the package because the driver and monitor classes declare virtual interface types. List only the package, never the individual class files. `+incdir` lets the package find them, and listing them again causes redefinition errors.

5. Interfaces and testbench top

AXI_BUS is your interface. The AXI driver uses its `Master` modport.

APB_BUS has three modports. `Master` is the bridge side. `Slave` is used by the APB driver, which reads the request and drives PREADY, PRDATA and PSLVERR. `Monitor` has only inputs. `clk` and `rst_n` were added to every modport, because a class holding a virtual modport cannot read a signal the modport does not list.

Testbench top (`tbench_top`):

- Clock: `always #5 clk = ~clk`.
- Reset: held low for 5 clocks, then released at a falling edge. The original code (`reset = 1; #5 reset = 0;`) treated an active-low reset as active-high, so it held the DUT in reset for the whole run. That was the root cause of the first timeout.
- It instantiates both interfaces and the DUT.
- It places three virtual interfaces in `config_db`: key `vif` (`AXI_BUS.Master`) for the AXI driver, `apb_slv_vif` (`APB_BUS.Slave`) for the APB driver, and `apb_mon_vif` (`APB_BUS.Monitor`) for the APB monitor. Then it calls `run_test()`.

6. AXI agent

6.1 Transaction: axi_seq_item

One item is one AXI request: a write, a read, or both together.

- **AW fields:** awvalid, awid (6 bits), awlen, awsize, awaddr, awburst, awuser (6 bits).
- **W fields:** wvalid, wdata, wstrb.
- **AR fields:** arvalid, arid, araddr, arlen, arsize, arburst, aruser.
- **Response fields** (filled by the driver after the transaction): bid, bresp, buser, rid, rdata, rresp, rlast, ruser.
- **Timing knobs** in clock cycles: aw_delay, w_delay, b_delay, r_delay. With aw_delay = 0 and w_delay > 0, AW goes first. The reverse puts W first. b_delay and r_delay hold BREADY and RREADY low.

Logic worth knowing:

- awvalid, wvalid and arvalid are `rand`, so every sequence pins them. A write sets awvalid and wvalid, a read sets arvalid, and setting all three tests simultaneous requests. The driver decides what to do from these flags.
- The default constraints (burst FIXED, sizes 1/2/4 bytes, address 0 to 127, lengths 1/2/4/8/16) are `soft`, so a sequence can override them to probe unsupported or boundary behaviour. Delays default to 0 through soft constraints.

- `do_copy` is implemented so `clone()` returns a real copy. Without it the clone is empty and coverage would read all zeros.

6.2 Sequencer and agent

`axi_sequencer` is a plain sequencer. `mem_agent` creates the driver and sequencer when active, connects them, and exposes `ap`, the driver's analysis port, for coverage.

6.3 Driver: axi_driver

Main loop: reset all signals, wait for `rst_n` to be 1, then forever get an item, run it, publish a clone, and call `item_done`.

Dispatch: awvalid and arvalid both set forks a write and a read. Otherwise awvalid runs a write, arvalid runs a read, and neither gives a warning.

Timing convention: the driver changes signals at the falling clock edge and samples at the rising edge, so the DUT never sees a race at the sampling edge.

Write path. `write_transaction` forks `drive_aw` and `drive_w`, waits for both, then runs `drive_b`.

- `drive_aw` and `drive_w` wait the configured delay, drive the fields and VALID at a falling edge, wait for READY at a rising edge, and drop VALID at the next falling edge. They run in parallel because the bridge accepts a write only when both channels are valid together. Waiting for AWREADY before raising WVALID could deadlock.
- `drive_b`, when b_delay is above 0, holds BREADY low for that many cycles. During that time it checks that BVALID stays high once seen and that BID, BRESP and BUSER do not change, as AXI requires. Then it raises BREADY, waits for BVALID, captures bid, bresp and buser, and drops BREADY.

Read path. `read_transaction` drives AR, optionally holds RREADY low for r_delay cycles with the same stability check on RVALID and its payload, then raises RREADY, waits for RVALID, and captures rid, rdata, rresp, rlast and ruser.

Safety nets.

- Every handshake wait has a 1000-cycle timeout. On timeout it raises a `uvm_error` naming the channel, instead of letting the test die with `PH_TIMEOUT`. The BVALID message points at the usual causes: PREADY not driven, or reset not released.
- Sideband signals the sequences do not use (lock, cache, prot, qos, region, user) are driven to 0 so the DUT never sees X.

- `publish_item` clones the finished request and writes it to `item_done_ap`.

7. AXI sequences

All are in `axi_sequence.sv`. Your two original sequences, `direct_write_seq` and `direct_read_seq`, are unchanged and still used by the virtual sequences. The rest use `uvm_do_with`.

Common rules:

- Every `uvm_do_with` pins awvalid, wvalid and arvalid.
- Functional sequences use awlen = arlen = 0 because the bridge is single-beat.
- Sequence knobs have names that cannot collide with item fields (`wr_addr`, `rd_id` and so on), so names in an inline constraint resolve unambiguously.

The base class `axi_base_seq` supplies the checkers:

- `check_wr`: BID must equal AWID and BUSER must equal AWUSER (errors). BRESP not OKAY is only a warning, because the RTL is documented to return OKAY.
- `check_rd`: RID = ARID, RUSER = ARUSER and RLAST = 1 (errors). RRESP not OKAY is a warning.
- `compare_data`: reports a mismatch between expected and received data.

The sequences:

- **axi_single_wr_seq:** one directed write with every knob (ID, address, data, strobe, USER, AW/W/B delays). Covers the AW and W handshakes, write conversion to APB, the B channel and BREADY back-pressure.
- **axi_single_rd_seq:** one directed read with ID, address, USER and R delay. It can compare RDATA with an expected value.
- **axi_wr_rd_back_seq:** writes random data, reads the same address back and compares. Proves write conversion and PRDATA capture end to end.
- **axi_aw_w_order_seq:** for gaps of 1, 2 and 5 cycles, writes with AW first then reads back, and writes with W first then reads back. Shows the write is accepted only when both channels are valid.
- **axi_wstrb_seq:** writes a full word with eight strobe patterns and reads each back. It logs an OBSERVED line saying whether WSTRB changed the data, which documents the unused WSTRB.
- **axi_bready_seq:** eight writes with BREADY delayed 1 to 8 cycles.

- **axi_rready_seq:** writes known data, then does eight reads with RREADY delayed 1 to 8 cycles, comparing data each time.
- **axi_id_user_seq:** IDs and USERs of 0, 63 and random across 12 write and read pairs. The checkers compare B and R IDs and USERs with the request.
- **axi_back_to_back_seq:** N writes, then N reads, then alternating write and read, with a reference memory.
- **axi_rand_mem_seq:** 30 random aligned writes with random delays, then reads every written address and compares against a reference memory.
- **axi_boundary_seq:** addresses from 0 up to 0xFFFFFFFC with data patterns. The expected value is tracked under the address masked to `apb_addr_width`, so addresses that differ only above the APB width are expected to alias. Set that knob to the DUT parameter.
- **axi_rd_wr_same_time_seq:** preloads an address with old data, then presents AW, W and AR together to that address with new data. If the read returns old data, read priority is confirmed. If it returns new data, the sequence raises an error. It then checks the write was not lost, and repeats with different addresses.
- **axi_unsupported_probe_seq:** sweeps awlen and arlen of 1/3/7/15 against FIXED/INCR/WRAP, and sizes 1/2/4 at byte offsets 0 to 3. It judges nothing. It logs OBSERVED lines for the report. The driver sends one W beat, which is all the bridge looks at.

8. APB agent

8.1 Why a slave driver

The bridge starts every APB transfer, so the APB agent never generates traffic. Its driver is a reactive responder and its monitor observes. A passive agent alone would hang the DUT.

8.2 Item and configuration

`apb_seq_item` holds the request fields observed on the bus (paddr, pwdata, pwrite) and the randomized response fields: `wait_cycles` (PENABLE cycles with PREADY low, where 0 means PREADY in the first cycle), `pslverr`, `prdata` and `override_prdata`. The monitor fills `observed_waits`. Soft constraints keep the defaults sane.

`apb_slave_cfg` holds the knobs: `is_active`, `min_wait` and `max_wait` for the default response (0 to 3), `err_pct` (percentage of transfers answered with PSLVERR), `default_rdata` (0xDEADBEEF for never-written addresses), `store_on_error`, `checks_en`, `strict_apb` (flags a missing setup phase, off because this RTL has none) and `pready_timeout` (256 cycles).

8.3 Slave driver: apb_slave_driver

- **Run loop:** idle the bus, wait for reset release, then race `serve_requests` against `@(negedge rst_n)`. An asynchronous reset therefore aborts any transfer in progress. The driver closes any open sequencer item, returns to idle and starts over.
- **Detecting a transfer:** at each rising edge plus 1 time unit (so DUT outputs have settled), if PSEL and PENABLE are both 1, a transfer has started. It captures PADDR, PWRITE and PWDATA.
- **Choosing the response:** `get_response` calls `try_next_item`. If a sequence has an item waiting, it is used. Otherwise a default item is randomized from the cfg ranges. This fallback is why the bridge cannot hang just because nobody started an APB sequence.
- **Wait states:** PREADY stays low for `wait_cycles` more cycles. Then, in one cycle, it drives PRDATA (0 for writes; for reads the item's value if `override_prdata`, else the memory or `default_rdata`), drives PSLVERR and raises PREADY.
- **Completion:** the next rising edge is the completion edge. Writes are stored in the memory model unless PSLVERR is set and `store_on_error` is off. One time unit later PREADY, PSLVERR and PRDATA return to idle. The finished request is written back into the item so a reactive sequence can inspect it, and `item_done` is called.
- **Why drive 1 time unit after the edge:** the response becomes visible in the same cycle without a race, which is what makes zero-wait-state transfers possible.
- **Backdoor:** `poke` and `peek` give tests direct access to the memory model.

An earlier version triggered on PSEL=1 with PENABLE=0. Because this bridge keeps PSEL high, that condition was true on every idle cycle. The driver then answered phantom reads of address 0 (returning 0xDEADBEEF) and missed the real writes. Triggering on PENABLE fixed it.

8.4 Monitor: apb_monitor

- A transfer is a run of cycles with PSEL and PENABLE high, ending at the first rising edge where PREADY is high. This works for a standard APB master and for this bridge.

- A two-state machine (IDLE, ACCESS) captures address, direction and write data on the first access cycle and counts the transfer. Each further cycle with PREADY low increments a wait counter. When PREADY is 1 it builds an `apb_seq_item` (address, data, PRDATA, PSLVERR, observed_waits) and writes it to `item_collected_port`.
- Checks, switchable with `checks_en`: PENABLE without PSEL; PSEL or PENABLE dropped before PREADY; address, direction or write data changing while waiting; PREADY stuck low beyond the timeout (reported once); X on PREADY or PSLVERR; and, only with `strict_apb`, a missing setup phase.
- It counts transfers that started without a setup phase and prints an OBSERVATION line in `report_phase`. That is a deviation from AMBA APB to record in the verification report, not a testbench error.
- A reset aborts the current transfer silently.

8.5 Response sequences

`apb_slave_base_seq` loops creating an item and randomizing it through an overridable `randomize_item`. A `num_responses` of 0 means run forever, so start it in the background and kill it at the end. Each item answers the next APB transfer. Subclasses:

- `apb_slave_rand_wait_seq`: wait states in a chosen range, no errors.
- `apb_slave_slverr_seq`: PSLVERR=1 on every response.
- `apb_slave_rdata_seq`: a fixed PRDATA (default 0xCAFEF00D) that overrides the memory.

8.6 Agent

`apb_slave_agent` always builds the monitor. It builds the driver and sequencer only when active. It reads `apb_slave_cfg` from `config_db`, shares it with its children, and re-exports the monitor's analysis port as `ap`.